

Object detection

ANN NIBANA S L

1 Dataset Preparation and Conversion

- Downloaded **DOTA-v1.0** dataset and extracted image and labelTxt.
- Selected **40 images** with aeroplane using a Python script.
- Used **dotadevkit** to convert the dataset to **COCO format** → generated DOTA_1.0.json.
- Split the dataset:
 - 30 images to images/train
 - 10 images to images/validation
- Created corresponding **COCO-style JSON** files: instances_train.json, instances_val.json

2. Folder Structure and Configuration

Organized dataset as:

mini_train/

├─ images/

| ├─ train/

| └─ val/

├─ labels/

| ├─ train/

| └─ val/

├─ annotations/

| ├─ instances_train.json

| └─ instances_val.json

Created a data.yaml file

3. YOLOv8 Setup, Label Fixes & Training

- Installed **Python 3.9** and YOLOv8 using pip install ultralytics.
- Resolved:
 - CA certificate error
 - CUDA compatibility (GPU verified)
- Fixed class ID issues in label .txt files (converted all class IDs to 0).
- Resized all training/validation images to **640×640** using PIL.
- Cleaned old cache files to prevent label mismatches.
- Trained YOLOv8 for **10 epochs**.

4. Inference, Output

- Ran predictions using the trained model (best.pt) on single and multiple validation images.
- Created a **custom output directory** using:
 `project="mini_train" name="custom_output"`
- Suppressed label text (aeroplane) in predictions using YOLO's Python interface with `show_labels=False`.

